AI 통제형 의사결정지원시스템
통합 기술설계서

추가 반영본 - 코 5 및 코 312 완료 문서 통합

문서 목적	기존 기술개발계획서, API 설계서, Docker/PostgreSQL/pgvector/Azure OpenAI 반영본에 Spring Boot 분리형 실서비스 구조와 관리자 UI 구성을 추가 통합
이번 추가 반영	코 5: Spring Boot + FastAPI 분리형 아키텍처, 서비스 간 통신, HITL 연계 구조 코 312 완료: 관리자 UI React 프로젝트, 대시보드/검토큐/감사로그 화면 구조
핵심 판단	현재 문서는 "아이디어 설명서" 수준을 넘어 "MVP-실서비스 전환 설계서" 수준으로 상향되었으며, 정부과제 제출 시 기술성·구현가능성 설명에 직접 활용 가능
권장 사용 방식	본 문서를 기술 파트 마스터본으로 사용하고, 예창패 제출본은 본 문서에서 2~4 쪽 압축본으로 별도 파생

1. 통합 반영 범위

기존 반영본에 아래 항목을 추가 통합했습니다.

소스 문서	반영 핵심	최종 적용 방식
기술서 122	기술개발계획의 기본 골격	문서의 본문 구조와 문제정의, 목표, 기대효과의 기준축 유지
Api 코파일럿	업무계-분석계 API 명세	요청/분석/정책/검토/감사 흐름을 표준 REST 구조로 통합
코 1~코 4	Docker Compose, Azure OpenAI, pgvector RAG	MVP 실행 패키지와 실사용 전환 포인트로 반영
코 5	Spring Boot + FastAPI 분리형 아키텍처	업무계와 AI 분석계를 서비스 단위로 분리한 실서비스 구조로 상향
코 312 완료	관리자 UI React 프로젝트	검토 큐, 대시보드, 요청 상세, 감사로그 화면을 운영계층으로 추가

2. 최종 목표 시스템 정의

최종 목표 시스템은 생성형 AI 를 직접 결정권자로 사용하는 구조가 아니라, 다음 5 개 축을 결합한 통제형 의사결정지원 플랫폼입니다.

- 업무 시스템 계층(Java/Spring Boot) - 요청 생성, 인증·권한, 검토 워크플로, 관리자 API 담당
- AI 분석 계층(Python/FastAPI) - RAG 검색, Azure OpenAI 호출, 위험도·불확실성 평가 담당
- 데이터 계층(PostgreSQL + pgvector) - 요청·분석·정책·감사 로그와 문서 임베딩 저장
- 운영 보조 계층(Redis, Docker Compose, 향후 Kubernetes) - 캐시, 비동기 큐, 배포 자동화
- 운영 UI 계층(React Admin UI) - 대시보드, 검토 대기열, 요청 상세, 감사로그 조회 담당

3. 추가 반영 후 최종 아키텍처

사용자/실무자
→ Spring Boot 업무계
 - OAuth2/JWT 인증
 - 요청 생성/조회
 - 승인/수정/반려(HITL)
 - 관리자 검토 API
→ FastAPI AI 분석계
 - pgvector RAG 검색
 - Azure OpenAI 호출
 - 위험도·불확실성 계산
 - 정책 판정 및 감사 이벤트 생성
→ PostgreSQL/pgvector
 - 요청·분석·근거·정책·감사 로그 저장
→ React 관리자 UI
 - 대시보드
 - 검토 큐
 - 요청 상세
 - 감사 로그 조회

4. 핵심 구현 스택 - 반영 결과

계층	권장 기술	역할	상태
업무 API	Spring Boot	요청 생성, 검토 워크플로, 외부 인증, 관리자 API	추가 반영 완료
AI 분석	FastAPI + SQLAlchemy	RAG, LLM 호출, 정책 판정, 감사 이벤트	반영 완료
LLM	Azure OpenAI	대화·요약·근거 포함 응답 생성	실서비스 연동안 반영
벡터 검색	pgvector	문서 임베딩 저장 및 유	실서비스 전환안 반영

		사도 검색	
주 DB	PostgreSQL	요청·분석·검토·정책·감사 로그 저장	반영 완료
캐시/큐	Redis	비동기 처리, 캐싱, 향후 확장	선택 구성으로 반영
운영 UI	React	대시보드, 검토큐, 감사 로그 화면	추가 반영 완료
배포	Docker Compose	개발/시연 환경 일괄 기동	반영 완료

5. 이번 추가 반영으로 올라간 완성도

평가 항목	이전 수준	현재 수준	판단 근거
기술 개요/문제정의	상	상	변경 없음. 기존 구조 유지
API/DB 설계	중상	상	요청-분석-정책-검토-감사 흐름이 구체화됨
실행 가능성	중	상	Docker Compose, PostgreSQL, pgvector, Azure OpenAI 반영
실서비스 구조	중하	상	Spring Boot 와 FastAPI 를 분리하여 업무계/AI 계 경계가 명확해짐
운영 관점 완성도	하	중상	관리자 UI 구조가 추가되어 실제 사용 시나리오가 보임

6. 관리자 UI 반영 요약

추가 업로드 문서 기준으로 운영자용 React UI 는 다음 4 개 화면이 기본 세트로 정의됩니다.

화면	핵심 기능	평가상 의미
Dashboard	리스크 분포, 총 요청 수, 자동 처리/검토 건수 시각화	운영 통제 가능성 제시
Pending Queue	검토 대기 목록, 페이지네이션, 요청 상세 진입	HITL 실무 흐름 제시
Request Detail	AI 초안, 근거 문서, 검토 의견, 최종 상태 확인	설명가능성/검토 가능성 제시

Audit Logs	행위자, 액션, 이유, 시각 조회	감사 가능성 제시

7. 제출용 핵심 문장 - 수정 반영본

1. 본 과제는 생성형 AI 자체의 성능 경쟁이 아니라, 공공·기업 환경에서 안전하게 운용 가능한 통제형 AI 업무지원 구조를 구현하는 데 목적이 있다.
2. 업무계는 Spring Boot 기반으로 구성하여 인증·권한·검토 워크플로와 같은 엔터프라이즈 기능을 안정적으로 처리하고, AI 분석계는 FastAPI 기반으로 분리하여 RAG·정책·감사 기능을 독립적으로 운영한다.
3. 문서 검색 계층은 PostgreSQL 과 pgvector 를 활용한 벡터 검색 구조를 채택하여 내부 규정과 지침을 근거로 활용하도록 설계한다.
4. AI 결과는 자동 집행되지 않으며, 위험도에 따라 자동 추천, 실무자 검토, 관리자 승인, 차단으로 분기되는 Human-in-the-Loop 구조를 기본 원칙으로 한다.
5. 관리자 UI 는 대시보드, 검토 대기열, 요청 상세, 감사로그 화면으로 구성하여 실제 운영 시나리오를 검증 가능한 수준으로 제시한다.

8. MVP 범위와 후속 확장 범위 분리

구분	범위	우선순위
즉시 MVP	FastAPI + PostgreSQL + Docker Compose + 기본 관리자 UI 연결	1
1 차 고도화	Azure OpenAI 실제 호출, pgvector 실검색, Redis 캐시	2
2 차 고도화	Spring Boot 업무계 분리 배포, mTLS, OPA 연동	3
3 차 고도화	Kubernetes, 모니터링 대시보드, 운영 지표 자동화	4

9. 최종 권고

현재 기준 최선의 전략은 “과장된 미래 서사”를 완전히 배제하고, 다음 순서로 문서를 잠그는 것입니다.

- 기술개발계획서 본문은 본 문서를 기준으로 유지
- 예창패 제출본은 본 문서에서 2~4 쪽 압축본으로 별도 제작
- 시연자료는 Docker Compose 패키지 + Swagger + 관리자 UI 목업/초기 화면으로 구성
- 평가위원 질의 대응 포인트는 ‘왜 Spring Boot 와 FastAPI 를 분리했는가’, ‘왜 HITL 이 필수인가’, ‘왜 관리자 UI 가 필요한가’ 세 축으로 통일

작성 기준일: 2026-03-12